

B.S. 4 Years Program / Fourth Semester - Spring 2024

Paper: Database Systems Course Code: CC-215 Time: 3 Hrs. Marks: 60
THE ANSWERS MUST BE ATTEMPTED ON THE ANSWER SHEET PROVIDED

Q.1. Answer the following short questions: (6x5=30)

a) Define entity and attribute of an entity. Briefly explain the difference between base attribute and derived attribute of an entity with the help of an example.

- **Entity:** In the context of database systems, an entity is a real-world object or concept that is distinguishable from other objects. It represents a "thing" about which data can be stored. Examples include a person, place, event, or object. For instance, in a university database, "Student," "Course," and "Professor" could be entities.
- **Attribute:** An attribute is a characteristic or property that describes an entity. It represents a piece of information about the entity. For example, for a "Student" entity, attributes might include "StudentID," "Name," "Address," and "DateOfBirth."
- **Base Attribute (Stored Attribute):** A base attribute is an attribute whose value is directly stored in the database. Its value is explicitly provided and does not depend on other attributes within the database for its calculation.
 - **Example:** For a "Student" entity, "DateOfBirth" and "FirstName" are base attributes, as their values are directly stored.
- **Derived Attribute:** A derived attribute is an attribute whose value is not directly stored in the database but can be computed or derived from other stored attributes. These attributes are often created to avoid data redundancy and maintain data consistency.
 - **Example:** For a "Student" entity, "Age" can be a derived attribute. It is not stored directly but can be calculated from the "DateOfBirth" (a base attribute) and the current date. Similarly, "TotalMarks" for a student might be derived by summing up marks from individual subjects.

b) Briefly explain data dictionary and its contents.

- **Data Dictionary:** A data dictionary is a central repository of information about data, such as meaning, relationships to other data, origin, usage, and format. It is a crucial component of a database management system (DBMS) that stores metadata (data about data). Essentially, it's a catalog of all the data items in a database system.
- **Contents of a Data Dictionary:** The data dictionary typically contains information about:
 - **Tables/Relations:** Names, owners, creation dates, number of rows.
 - **Columns/Attributes:** Names, data types (e.g., INT, VARCHAR, DATE), lengths, precision, nullability constraints (whether the column can contain NULL values), default values, primary key/foreign key status.
 - **Indexes:** Names, columns indexed, type of index (e.g., B-tree, hash).
 - **Views:** Definitions of virtual tables.
 - **Stored Procedures/Functions:** Code, parameters, return types.
 - **Triggers:** Event, timing, and action.
 - **Users and Permissions:** User accounts, roles, and privileges (who can access what).

- **Constraints:** Primary key, foreign key, unique, check, and not null constraints.
- **Relationships:** How tables are linked (e.g., one-to-many, many-to-many).

c) What is a foreign key? How it helps in maintaining the data integrity and data quality?

- **Foreign Key:** A foreign key is a column or a set of columns in a database table that refers to the primary key of another table. It establishes a link or relationship between two tables. The table containing the foreign key is called the "child table," and the table containing the primary key is called the "parent table."
- **How it helps in maintaining data integrity and data quality:**
 - **Referential Integrity:** The primary role of a foreign key is to enforce referential integrity. This means that if a foreign key value exists in the child table, it *must* have a matching primary key value in the parent table, or it must be NULL (if the foreign key is allowed to be NULL). This prevents "orphan records" – child records that refer to non-existent parent records.
 - **Data Consistency:** By ensuring referential integrity, foreign keys maintain data consistency across related tables. For example, if you have an Orders table and a Customers table, a foreign key linking CustomerID in Orders to CustomerID in Customers ensures that every order is placed by an existing customer.
 - **Prevents Deletion of Referenced Records:** Most DBMSs, when a foreign key is in place, will prevent the deletion of a record in the parent table if there are still matching records in the child table (unless specific ON DELETE actions like CASCADE or SET NULL are defined). This prevents broken links and ensures that crucial parent data isn't accidentally removed while child data still references it.
 - **Facilitates Data Relationships:** Foreign keys are the backbone of relational databases, allowing for the establishment of meaningful relationships between different entities, which is crucial for querying, reporting, and maintaining a well-structured database.
 - **Improved Data Quality:** By enforcing these rules, foreign keys contribute significantly to the overall quality of the data, reducing errors, inconsistencies, and anomalies.

d) Briefly explain distributed database management system (DDBMS).

- **Distributed Database Management System (DDBMS):** A DDBMS is a database system where the data is stored on multiple computers located in different geographical locations, but these locations are logically interconnected and managed as a single database system. To the end-user, a DDBMS appears as a single, centralized database, abstracting away the complexities of distribution. The data can be replicated or fragmented across different sites.
- **Key Characteristics and Concepts:**
 - **Distribution Transparency:** Users do not need to know where the data is physically stored; the DDBMS handles data localization.
 - **Replication Transparency:** Users are unaware if data is replicated across multiple sites; the DDBMS manages consistency among replicas.
 - **Fragmentation Transparency:** Data can be divided into fragments (horizontal or vertical) and stored at different sites, without the user being aware of this division.
 - **Concurrency Control:** Managing simultaneous access to distributed data to ensure data integrity and consistency.
 - **Distributed Query Processing:** Optimizing queries that involve data spread across multiple sites.
 - **Distributed Transaction Management:** Ensuring atomicity, consistency, isolation,

- and durability (ACID properties) for transactions spanning multiple nodes.
- **Heterogeneity:** A DDBMS can integrate different types of database systems (e.g., Oracle, SQL Server, MySQL) at various sites.
- **Autonomy:** Each site in a DDBMS can maintain a degree of independence in its operations, even while participating in the distributed system.

e) What is the use of locking in concurrent transactions? Discuss different levels (granularity) of a lock. Briefly explain COMMIT, ROLLBACK, and SAVEPOINT commands for transaction management.

- **Use of Locking in Concurrent Transactions:**

- Locking is a mechanism used in database systems to manage concurrent access to shared data. Its primary purpose is to ensure data integrity and consistency when multiple transactions are trying to read or modify the same data simultaneously.
- Without locking, issues like **lost updates** (one transaction's update overwritten by another), **dirty reads** (reading uncommitted data), **non-repeatable reads** (reading the same data twice but getting different values due to another transaction's commit), and **phantom reads** (new rows appearing in a range query) can occur.
- Locks prevent these anomalies by temporarily restricting access to data items.

- **Different Levels (Granularity) of a Lock:**

- Lock granularity refers to the size of the data item that is locked. A coarser granularity means a larger data item is locked, while a finer granularity means a smaller data item is locked.
- **Database-Level Lock:** Locks the entire database. This is the coarsest granularity. While simple to implement, it severely limits concurrency as only one transaction can access the database at a time. Rarely used in production for general transactions.
- **Table-Level Lock:** Locks an entire table. All rows in the table are locked. This is coarser than row-level and can reduce concurrency if many transactions need to access different rows in the same table. It's often used for DDL operations (e.g., ALTER TABLE) or bulk updates.
- **Page-Level Lock:** Locks a data page (a fixed-size block of data on disk). A page can contain multiple rows. If a transaction needs to access a row on a page, the entire page is locked. This offers better concurrency than table-level but can still lead to contention if multiple transactions need rows on the same page.
- **Row-Level Lock:** Locks individual rows. This is the finest granularity and offers the highest level of concurrency, as transactions can work on different rows within the same table simultaneously. However, managing many row-level locks can incur higher overhead.
- **Field-Level (Column-Level) Lock:** Locks individual fields or columns within a row. This is even finer granularity but is rarely implemented due to the high overhead and complexity involved in managing such granular locks.

- **Transaction Management Commands:**

- **COMMIT:** The COMMIT command is used to permanently save all the changes made during the current transaction to the database. Once a transaction is committed, the changes become permanent and visible to all other transactions. It signifies the successful completion of a transaction, making its operations durable.
- **ROLLBACK:** The ROLLBACK command is used to undo all the changes made during the current transaction. If a transaction encounters an error, or if the user decides to cancel the operations, ROLLBACK restores the database to the state it

was in before the transaction began. It ensures atomicity, meaning either all changes are applied, or none are.

- **SAVEPOINT:** A SAVEPOINT is a point within a transaction to which you can later roll back. It allows for partial rollbacks within a transaction without undoing the entire transaction. If an error occurs after a SAVEPOINT, you can roll back to that specific savepoint, leaving the changes made before the savepoint intact. This is useful for complex transactions where you might want to test intermediate steps or handle specific errors without restarting the entire transaction.

Q.2. Answer the following questions: (3x10=30)

i. Draw an ER diagram for an organization to keep track of all employees, their skills, projects assigned and their departments. Every employee has a unique number and each employee is given a job title. We are interested in collecting more data specific to engineers and secretary. The relevant data to be recorded for engineers is the type of degree and for secretaries is their typing speed. Many employees can work on a project and vice-versa. An employee can have many skills and they will use only a given set of skills on a particular project. There are 10 different departments each with a unique name. An employee can report to only one department. Each department is managed by a chairman. Each department deals with many vendors for purchasing various kinds of equipment. A vendor typically supplies equipment to many departments. It is required to store the details of each order delivered by a vendor to a certain department.

(Please note: Drawing an ER diagram here is challenging. I will describe the entities, attributes, and relationships in detail, which can then be used to draw the ER diagram.)

Entities and Attributes:

1. **EMPLOYEE** (Strong Entity)
 - EmpID (Primary Key, Unique Number)
 - EmpName
 - JobTitle
 - Salary (Assumed attribute)
 - DateOfHire (Assumed attribute)
 - DeptID (Foreign Key referencing DEPARTMENT)
2. **ENGINEER** (Weak Entity, Specialization of EMPLOYEE)
 - EmpID (Partial Key, also Foreign Key referencing EMPLOYEE)
 - DegreeType (e.g., BS, MS, PhD)
3. **SECRETARY** (Weak Entity, Specialization of EMPLOYEE)
 - EmpID (Partial Key, also Foreign Key referencing EMPLOYEE)
 - TypingSpeed (WPM)
4. **SKILL** (Strong Entity)
 - SkillID (Primary Key, Assumed)
 - SkillName (e.g., Java, Python, Project Management)
 - SkillDescription (Assumed)
5. **PROJECT** (Strong Entity)
 - ProjectID (Primary Key, Assumed)
 - ProjectName
 - StartDate
 - EndDate
 - Budget (Assumed)
6. **DEPARTMENT** (Strong Entity)
 - DeptID (Primary Key, Unique Name implies this is the primary key)

- DeptName (Unique Name)
- Location (Assumed)
- ChairmanID (Foreign Key referencing EMPLOYEE - an employee manages a department)
- 7. **VENDOR** (Strong Entity)
 - VendorID (Primary Key, Assumed)
 - VendorName
 - ContactPerson
 - Phone
 - Address
- 8. **ORDER** (Strong Entity) - Represents a specific order from a vendor to a department.
 - OrderID (Primary Key, Assumed)
 - OrderDate
 - DeliveryDate
 - TotalAmount
 - VendorID (Foreign Key referencing VENDOR)
 - DeptID (Foreign Key referencing DEPARTMENT)

Relationships:

1. **EMPLOYEE IS_A {ENGINEER, SECRETARY}**: This is a **specialization/generalization** (or ISA) relationship. An EMPLOYEE can be either an ENGINEER or a SECRETARY (or neither, if there are other job titles not specified). This is a disjoint (an employee cannot be both simultaneously) and partial (not all employees are engineers or secretaries) specialization.
 - Represented by a triangle connecting EMPLOYEE to ENGINEER and SECRETARY, with a "d" (for disjoint) inside the triangle.
2. **WORKS_FOR** (Between EMPLOYEE and DEPARTMENT)
 - Cardinality: Many-to-One (Many Employees work for One Department)
 - An Employee reports to *only one* Department.
 - Attributes: None (already captured by DeptID in EMPLOYEE)
3. **MANAGES** (Between EMPLOYEE and DEPARTMENT)
 - Cardinality: One-to-One (One Employee manages One Department, and One Department is managed by One Employee)
 - ChairmanID in DEPARTMENT table references EmpID in EMPLOYEE. This assumes a chairman is an employee.
4. **HAS_SKILL** (Between EMPLOYEE and SKILL)
 - Cardinality: Many-to-Many (Many Employees can have Many Skills)
 - Requires a **Junction Table** (e.g., EMPLOYEE_SKILL)
 - Attributes for EMPLOYEE_SKILL: EmpID, SkillID, ProficiencyLevel (e.g., Beginner, Intermediate, Expert - Assumed)
5. **WORKS_ON** (Between EMPLOYEE and PROJECT)
 - Cardinality: Many-to-Many (Many Employees can work on Many Projects, and Many Projects can have Many Employees)
 - Requires a **Junction Table** (e.g., EMPLOYEE_PROJECT)
 - Attributes for EMPLOYEE_PROJECT: EmpID, ProjectID, Role (e.g., Team Lead, Developer), HoursWorked (Assumed)
6. **USES_SKILL_ON_PROJECT** (Ternary Relationship between EMPLOYEE, SKILL, and PROJECT)
 - "An employee can have many skills and they will use only a given set of skills on a

particular project." This implies a ternary relationship or two binary relationships. A ternary relationship (Employee-Skill-Project) would capture the specific skills used by a specific employee on a specific project.

- Attributes: EmpID, SkillID, ProjectID. This might also be modeled as an attribute of EMPLOYEE_PROJECT for specific skills used *on that project*. Let's stick with the ternary for clarity as requested.

7. **SUPPLIES** (Between VENDOR and DEPARTMENT)

- Cardinality: Many-to-Many (Many Vendors supply to Many Departments, and Many Departments deal with Many Vendors)
- This relationship is captured by the ORDER entity, which directly links a VENDOR to a DEPARTMENT for a specific order. So, implicitly, through ORDER, VENDOR supplies to DEPARTMENT.

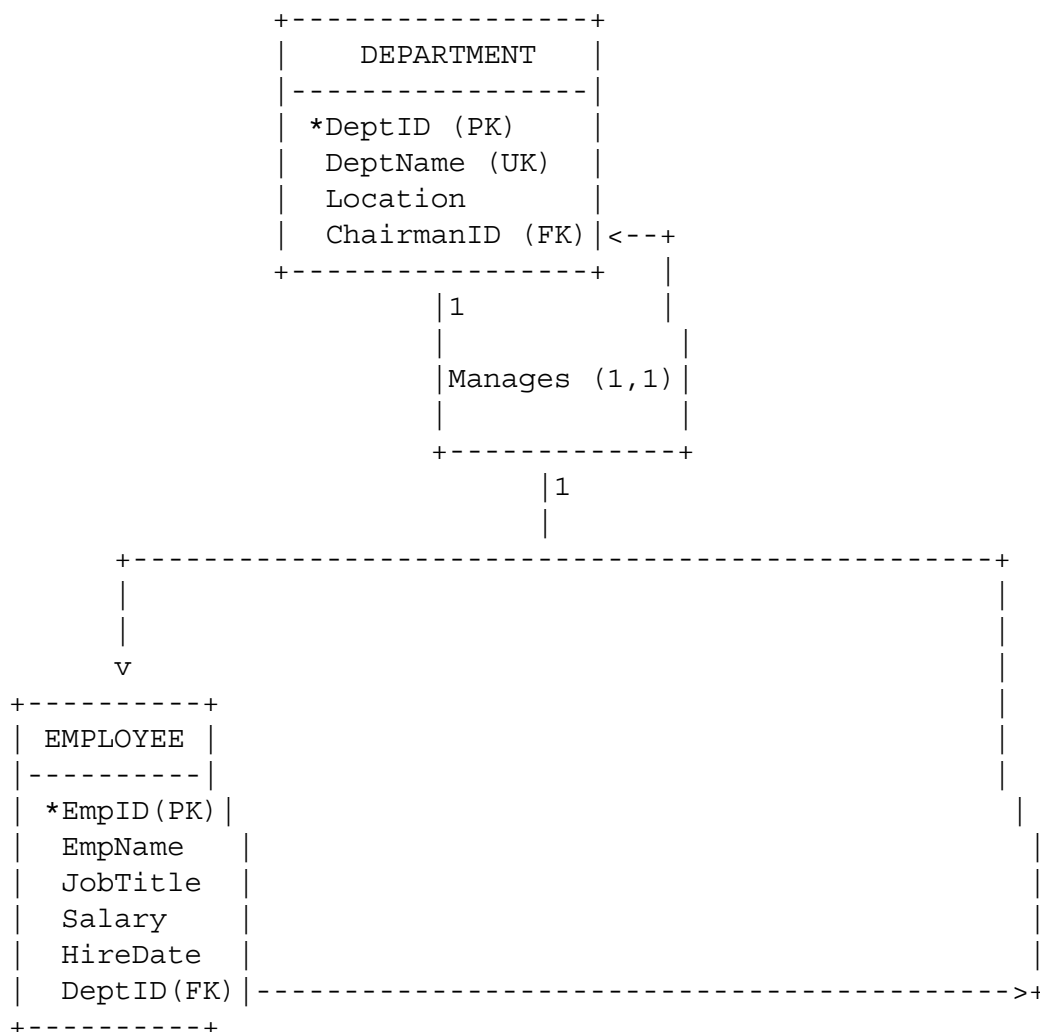
8. **PLACES_ORDER** (Between DEPARTMENT and ORDER)

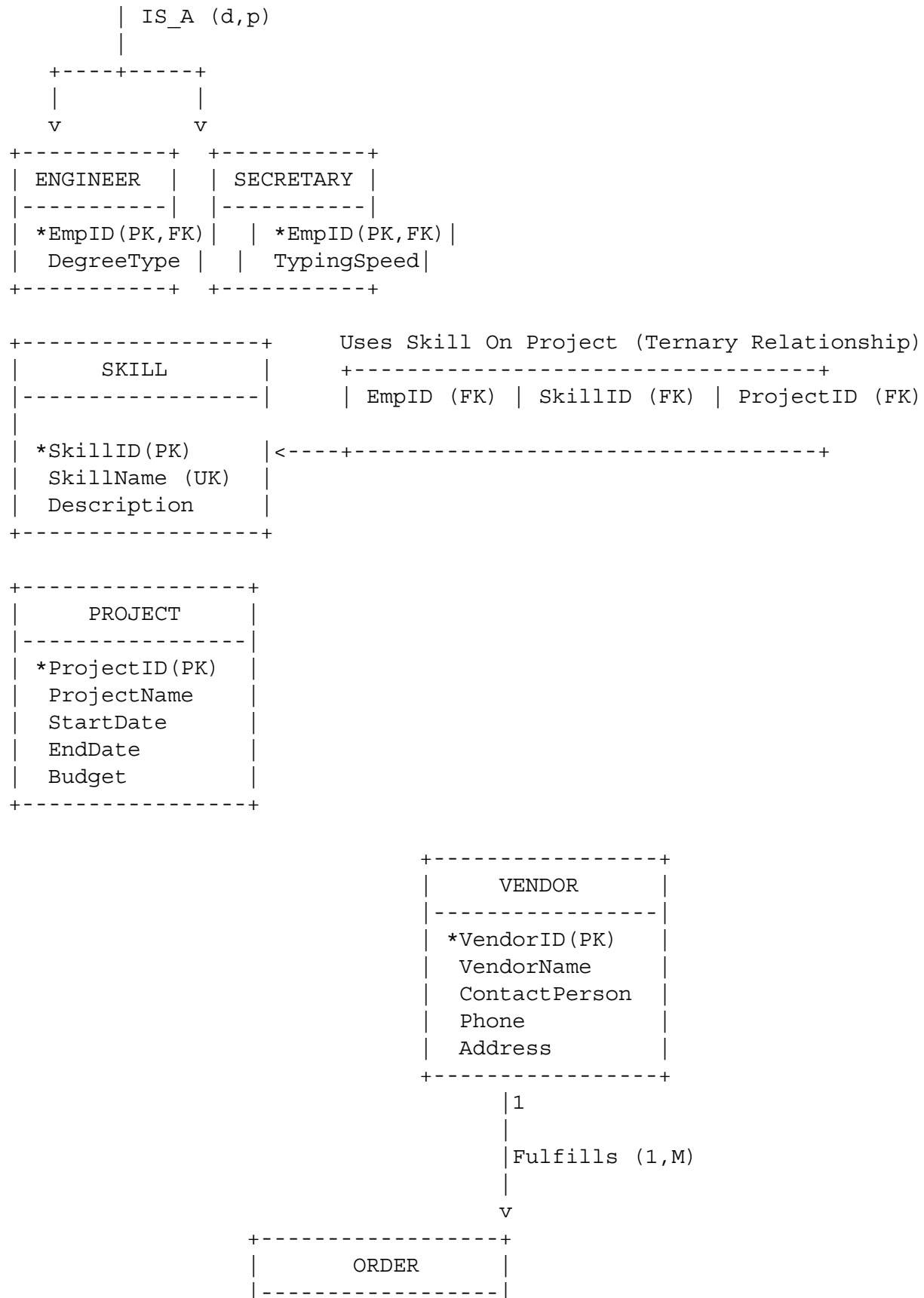
- Cardinality: One-to-Many (One Department can place Many Orders)
- Captured by DeptID in ORDER.

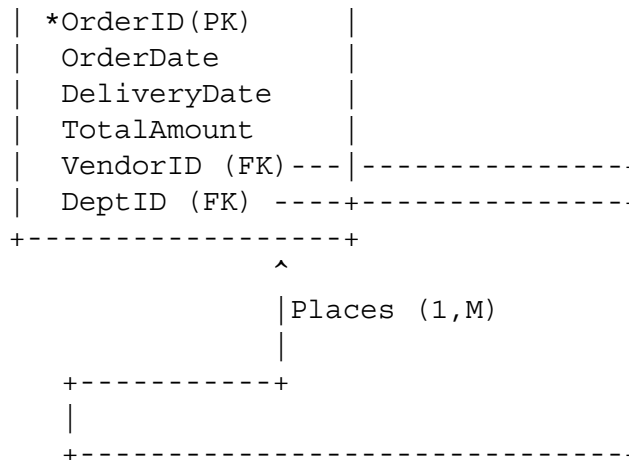
9. **FULFILLS_ORDER** (Between VENDOR and ORDER)

- Cardinality: One-to-Many (One Vendor can fulfill Many Orders)
- Captured by VendorID in ORDER.

ER Diagram Sketch (Conceptual):







(Please note: For the ER Diagram, the above text describes the entities, attributes, and relationships, including cardinality. A proper visual ER diagram would use standard ER notation with rectangles for entities, ovals for attributes, diamonds for relationships, and lines with cardinalities. The ternary relationship would be a diamond connected to the three entities.)

ii. Normalize the following relation step by step up to 3NF. You must specify the definition of the normal form and set of relations obtained after each Normal form.

Original Relation: StdID StdName CourseID Cr.Hrs. UnitCode UnitName Instructor
 001 Usman A203 3 U10 ICT Omer
 003 Ahmad A104 4 U11 Media Adnan
 007 Qamar A203 3 U10 ICT Nadeem
 007 Qamar A203 3 U11 DIP Zahid
 010 Fatima A323 2 U23 DBS Naseem
 010 Fatima A323 2 U33 DBA Asim

Let's identify functional dependencies (FDs) first. From the data, we can infer:

- StdID $\rightarrow$ StdName (001 is Usman, 003 is Ahmad, 007 is Qamar, 010 is Fatima)
- CourseID $\rightarrow$ Cr.Hrs. (A203 is 3, A104 is 4, A323 is 2)
- UnitCode $\rightarrow$ UnitName (U10 is ICT, U11 is Media, U21 is OOP, U31 is DBMS, U33 is DBA, U23 is DBS)
- (StdID, CourseID, UnitCode) $\rightarrow$ Instructor (An instructor is specific to a student, course, and unit combination. For example, (001, A203, U10) $\rightarrow$ Omer, (001, A203, U11) $\rightarrow$ Adnan)
- (StdID, CourseID) $\rightarrow$ UnitCode (A student in a course can have multiple units. Not a direct FD)
- (StdID, CourseID, UnitCode) $\rightarrow$ All attributes (This is the primary key for the original table)

Primary Key of the original relation: (StdID, CourseID, UnitCode) This is because StdID alone is not unique (001 appears multiple times), CourseID alone is not unique, UnitCode alone is not unique. Combined, they uniquely identify each row.

Step 1: First Normal Form (1NF)

Definition: A relation is in 1NF if and only if all attribute domains contain only atomic (indivisible) values, and there are no repeating groups of columns. Each column contains a single value, and there's a unique primary key.

Analysis: The given relation is already in 1NF because:

- Each cell contains a single value (e.g., Usman, A203, 3).
- There are no repeating groups.

- There is a clear primary key (StdID, CourseID, UnitCode).

Relations after 1NF: R1 (StdID, StdName, CourseID, Cr.Hrs., UnitCode, UnitName, Instructor)
Primary Key: (StdID, CourseID, UnitCode)

Step 2: Second Normal Form (2NF)

Definition: A relation is in 2NF if it is in 1NF and all non-key attributes are fully functionally dependent on the primary key. This means there should be no partial dependencies (where a non-key attribute depends on only a part of a composite primary key).

Analysis of Partial Dependencies: Primary Key: (StdID, CourseID, UnitCode)

1. StdID $\rightarrow$ StdName (Partial dependency: StdName depends only on StdID, which is a part of the composite primary key).
2. CourseID $\rightarrow$ Cr.Hrs. (Partial dependency: Cr.Hrs. depends only on CourseID, which is a part of the composite primary key).
3. UnitCode $\rightarrow$ UnitName (Partial dependency: UnitName depends only on UnitCode, which is a part of the composite primary key).

Normalization: We need to remove these partial dependencies by creating new relations.

Relations after 2NF:

- **Student Table (STUDENT)** to resolve StdID $\rightarrow$ StdName
 - STUDENT (StdID, StdName)
 - Primary Key: StdID
 - Data:
 - (001, Usman)
 - (003, Ahmad)
 - (007, Qamar)
 - (010, Fatima)
- **Course Table (COURSE)** to resolve CourseID $\rightarrow$ Cr.Hrs.
 - COURSE (CourseID, Cr.Hrs.)
 - Primary Key: CourseID
 - Data:
 - (A203, 3)
 - (A104, 4)
 - (A323, 2)
- **Unit Table (UNIT)** to resolve UnitCode $\rightarrow$ UnitName
 - UNIT (UnitCode, UnitName)
 - Primary Key: UnitCode
 - Data:
 - (U10, ICT)
 - (U11, Media)
 - (U21, OOP)
 - (U31, DBMS)
 - (U33, DBA)
 - (U23, DBS)
- **Enrollment/Study Table (ENROLLMENT)** to keep the remaining dependencies and the original primary key.
 - ENROLLMENT (StdID, CourseID, UnitCode, Instructor)
 - Primary Key: (StdID, CourseID, UnitCode)
 - Foreign Keys:
 - StdID references STUDENT(StdID)
 - CourseID references COURSE(CourseID)

- UnitCode references UNIT(UnitCode)
- Data:
 - (001, A203, U10, Omer)
 - (001, A203, U11, Adnan)
 - (003, A104, U21, Omar)
 - (003, A104, U31, Nadeem)
 - (007, A203, U10, Nadeem)
 - (007, A203, U11, Zahid)
 - (010, A323, U23, Naseem)
 - (010, A323, U33, Asim)

Step 3: Third Normal Form (3NF)

Definition: A relation is in 3NF if it is in 2NF and there are no transitive dependencies. A transitive dependency exists when a non-key attribute depends on another non-key attribute. (i.e., $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$ is a transitive dependency if B is not a superkey).

Analysis of Transitive Dependencies for ENROLLMENT (StdID, CourseID, UnitCode, Instructor):

- Primary Key: (StdID, CourseID, UnitCode)
- Non-key attribute: Instructor

From the data, it appears that $\text{UnitCode} \rightarrow \text{Instructor}$ *in some cases* (U10 has Omer, Nadeem; U11 has Adnan, Zahid; U21 has Omar; U31 has Nadeem; U23 has Naseem; U33 has Asim). However, Instructor actually depends on the combination of StdID, CourseID, and UnitCode. For instance, U10 is taught by Omer for (001, A203, U10) and by Nadeem for (007, A203, U10). This means UnitCode alone does not determine Instructor.

Let's re-examine the original data and the ENROLLMENT table. ENROLLMENT (StdID, CourseID, UnitCode, Instructor)

- $(\text{StdID}, \text{CourseID}, \text{UnitCode}) \rightarrow \text{Instructor}$ (This is the full functional dependency of the non-key attribute on the primary key, so no partial dependency.)

There are no other non-key attributes that Instructor could transitively depend on in the ENROLLMENT table. For example, we do *not* have a dependency like $\text{Instructor} \rightarrow \text{SomeOtherNonKeyAttribute}$.

Therefore, the ENROLLMENT table (and STUDENT, COURSE, UNIT tables) are already in 3NF.

Relations after 3NF:

- **STUDENT (StdID, StdName)**
 - Primary Key: StdID
- **COURSE (CourseID, Cr.Hrs.)**
 - Primary Key: CourseID
- **UNIT (UnitCode, UnitName)**
 - Primary Key: UnitCode
- **ENROLLMENT (StdID, CourseID, UnitCode, Instructor)**
 - Primary Key: (StdID, CourseID, UnitCode)
 - Foreign Keys:
 - StdID references STUDENT(StdID)
 - CourseID references COURSE(CourseID)
 - UnitCode references UNIT(UnitCode)

All relations are now in 3NF as they are in 2NF and have no transitive dependencies.

iii. Create the following tables and apply the specified constraints:

Table: Product | Attribute | Constraints || :----- | :----- || Prod# | Primary Key ||
 ProdName | Not Null || Price | Between 500 and 15000 || Stock | Greater than 0 |

SQL for Product Table:

```
CREATE TABLE Product (
    Prod# INT PRIMARY KEY,
    ProdName VARCHAR(255) NOT NULL,
    Price DECIMAL(10, 2) CHECK (Price BETWEEN 500 AND 15000),
    Stock INT CHECK (Stock > 0)
);
```

Table: Invoice | Attribute | Constraints || :----- | :----- || Invoice# | Primary Key || ItemID | Primary Key, Foreign Key || Prod# | Foreign Key || Qty_Ordered | Greater than 0 |

SQL for Invoice Table:

```
CREATE TABLE Invoice (
    Invoice# INT,
    ItemID INT, -- Assuming ItemID is part of a composite primary key
    with Invoice# to uniquely identify each item in an invoice
    Prod# INT,
    Qty_Ordered INT CHECK (Qty_Ordered > 0),
    PRIMARY KEY (Invoice#, ItemID),
    FOREIGN KEY (Prod#) REFERENCES Product(Prod#)
);
```

Correction/Clarification on Invoice Table: The prompt specifies ItemID as Primary Key, Foreign Key. This is ambiguous. If ItemID is a FK to another Item table, it can't be a PK *of this table* on its own unless Invoice is a weak entity to Item. A more common design for an Invoice and its items is:

- Invoice table (Invoice# PK)
- Invoice_Line_Item table (Invoice# PK, ItemID PK, Prod# FK, Qty_Ordered)

Given the structure "ItemID: Primary Key, Foreign Key", it implies ItemID is a foreign key *and* part of the composite primary key of Invoice (together with Invoice#). I've used this interpretation. If ItemID refers to a separate Item table, then that Item table would need to be created first, and ItemID would reference *its* primary key.

iv) Write SQL statements for the above database: (1.5 each)

a) Update the price of all the products by 15% of their current price, whose current price is between 100 and 5000.

```
UPDATE Product
SET Price = Price * 1.15
WHERE Price BETWEEN 100 AND 5000;
```

(Note: The Product table's check constraint Price BETWEEN 500 AND 15000 might conflict if prices after update go outside this range. Assuming the update is intended for the existing data within the valid range. The question specifies price between 100 and 5000, which extends beyond the Product table's constraint of 500-15000. I'll use the question's criteria for the WHERE clause.)

b) Display Invoice#, Prod#, ProductName, and Price for all the invoices generated in the year 2023.

```

SELECT
    I.Invoice#,
    I.Prod#,
    P.ProdName,
    P.Price
FROM
    Invoice I
JOIN
    Product P ON I.Prod# = P.Prod#
WHERE
    STRFTIME('%Y', I.InvoiceDate) = '2023'; -- For SQLite or similar
    -- Or YEAR(I.InvoiceDate) = 2023; -- For MySQL
    -- Or EXTRACT(YEAR FROM I.InvoiceDate) = 2023; -- For PostgreSQL
    -- Or DATEPART(YEAR, I.InvoiceDate) = 2023; -- For SQL Server

```

(Assumption: The Invoice table needs an InvoiceDate column for this query. I'll add this to the Invoice table definition for completeness in the solution.)

Revised SQL for Invoice Table (with InvoiceDate):

```

CREATE TABLE Invoice (
    Invoice# INT,
    ItemID INT,
    Prod# INT,
    Qty_Ordered INT CHECK (Qty_Ordered > 0),
    InvoiceDate DATE, -- Added InvoiceDate for query (b)
    PRIMARY KEY (Invoice#, ItemID),
    FOREIGN KEY (Prod#) REFERENCES Product(Prod#)
);

```

c) How many products have stock below 100?

```

SELECT
    COUNT(Prod#)
FROM
    Product
WHERE
    Stock < 100;

```

d) Display minimum, maximum, and average price using the product table.

```

SELECT
    MIN(Price) AS MinimumPrice,
    MAX(Price) AS MaximumPrice,
    AVG(Price) AS AveragePrice
FROM
    Product;

```